

Analysis of Quantization Methods in Transformer-Based Neural Networks

Sahil Bhatt

Research in Computer and Systems Engineering
Technische Universität Ilmenau
sahil.bhatt@tu.ilmenau.de

Abstract—Transformer models achieve strong results in language and vision tasks, but their parameter counts and memory demands complicate efficient deployment. This group study analyzes weight quantization in two transformer architectures—DistilBERT and ViT-B/16—with ResNet-18 as a CNN reference. It compares symmetric per-channel post-training quantization (PTQ) with tensor-local scalar codebooks, using floating-point reconstruction with FP32 activations and no quantization-aware training. Experiments use the complete CIFAR-10 test split and labelled SST-2 development split. Two-to-eight-bit sweeps measure accuracy and estimate parameter compression, while four- and eight-bit layer or module studies measure isolated sensitivity and cross-bit rank stability. At eight bits, the maximum accuracy degradation across all three models and both custom methods is 0.59 percentage points, with approximately 75% parameter-storage reduction. At four bits, DistilBERT and ViT remain within 0.6 percentage points of their FP32 baselines while achieving about $7.9\times$ compression. ResNet-18 instead falls from 90.47% to 11.56% with PTQ and 8.43% with codebook quantization; quantizing its first convolution alone causes losses of 60.65 and 78.62 percentage points. These observations concern the evaluated checkpoints and pipeline. ViT used a frozen backbone whereas ResNet was adapted end-to-end; training and checkpoint differences prevent isolating an architecture effect. The cause of the severe ResNet degradation remains unresolved.

Index Terms—neural-network quantization, post-training quantization, vector quantization, codebook quantization, layer sensitivity, ResNet, DistilBERT, Vision Transformer

I. INTRODUCTION

Deep neural networks achieve strong performance in computer vision and natural-language processing, but their parameter storage, memory traffic, and arithmetic requirements complicate deployment on mobile, embedded, and edge systems. Quantization addresses these costs by representing weights, activations, or both with fewer bits than FP32 [1]–[3]. Replacing 32-bit parameters by packed 8-bit or 4-bit values suggests ideal storage reductions of four or eight times. Actual savings are lower because scales, codebooks, alignment, and preserved parameters must also be stored. More importantly, low precision introduces numerical error that can change model decisions.

The resulting accuracy cost depends on the checkpoint, training history, quantization configuration and architecture. A convolutional network constructs hierarchical visual features through local kernels. A language transformer uses token embeddings, attention projections, and feed-forward matrices.

A vision transformer converts image patches into tokens before applying transformer encoders. These components differ in parameter distribution, function, and position in the inference path, and they need not tolerate the same quantization policy. A full-model accuracy number also does not reveal whether failure is distributed across the network or dominated by a small number of sensitive components.

This project studies two post-training, weight-only techniques. The first is symmetric uniform PTQ with per-output-channel scaling. The second is non-uniform codebook quantization, in which each weight is represented by a low-bit index into a learned set of centroids. Both are research-informed custom implementations: the code is original to this project, while the mathematical principles are based on established literature. No new quantization algorithm is claimed. Evaluation simulates low-bit weights through floating-point reconstruction with FP32 activations; no quantization-aware training is used.

This group study focuses primarily on quantization in transformer-based artificial neural networks. DistilBERT represents a language transformer and ViT-B/16 represents a vision transformer; ResNet-18 provides a CNN reference for a descriptive comparison of the selected checkpoints, rather than a controlled test of architecture alone. The experimental design combines full-model bit-width sweeps with isolated layer, tensor, or module experiments in which only one analysis unit is quantized while all remaining weights stay FP32. Accuracy is measured directly, while parameter-representation storage is estimated from value counts, bit widths, and required metadata. Computational efficiency is discussed only through parameter storage because the custom evaluation path reconstructs weights for floating-point Keras execution and does not benchmark latency, throughput, energy, or runtime memory.

A. Research Questions

The study addresses the following questions:

- 1) How do bit width and quantization method affect accuracy, compression ratio, and parameter-memory reduction?
- 2) How do the selected DistilBERT, ViT-B/16 and ResNet-18 checkpoints respond to four- and eight-bit quantization under the evaluated policies, given their different training procedures?
- 3) Which layers, tensors, or modules cause the largest accuracy degradation when quantized in isolation?

- 4) Does an eight-bit sensitivity ranking reliably predict four-bit sensitivity?

B. Scope and Deliverables

The project scope and principal outputs are:

- reusable mathematical n -bit PTQ and codebook weight-quantization implementations with explicit storage accounting;
- a common quantization and evaluation framework applied to three separately trained checkpoints;
- complete full-evaluation-split two-to-eight-bit PTQ and codebook sweeps for all three architectures;
- exhaustive W8 and exhaustive or architecture-stratified W4 sensitivity studies across convolutional and transformer models;
- reproducible notebooks with fixed selection policies, cached predictions, CSV tables, figures, and run meta-data; and
- a checkpoint-specific comparison that documents measured accuracy changes and the limits of attributing them to architecture.

II. TRANSFORMER ARCHITECTURES, MODELS, AND DATASETS

The evaluation covers three model families and two data modalities. Each quantized model is derived from the same saved task-specific FP32 checkpoint used for its baseline. Accuracy degradation is therefore measured relative to the FP32 version of the same architecture and task, not by comparing absolute accuracies across different datasets.

A. Transformer Architecture Context

A transformer encoder represents its input as a sequence of token embeddings and repeatedly applies multi-head self-attention [4] and a position-wise feed-forward network. For one attention head, input projections form query, key, and value matrices Q , K , and V , and scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad (1)$$

where d_k is the key dimension [4]. Multiple heads learn different projection subspaces, after which their outputs are concatenated and projected. Residual connections, normalization, positional information, and feed-forward layers complete each encoder block. The large embedding, attention-projection, and feed-forward matrices are therefore natural targets for weight quantization.

DistilBERT converts subword tokens into learned token and position embeddings and processes them with six transformer layers for sentence classification. ViT-B/16 instead converts image patches into token embeddings, prepends a learned class token, adds positional embeddings, and processes the resulting sequence with 12 transformer blocks. Although their input modalities differ, both models rely primarily on the same attention and feed-forward matrix operations. This shared structure enables a consistent quantization study, while

ResNet-18 provides a convolution-based reference with local kernels and hierarchical spatial features.

B. ResNet-18 and CIFAR-10

ResNet-18 is built from residual blocks whose shortcut connections allow a block to learn a residual transformation relative to its input [5]. It is selected as the CNN reference because its progression from an input convolution through residual stages to a classification head provides clearly defined kernels for layer-wise analysis. The model was initialized from the KerasHub `resnet_18_imagenet` preset and adapted end-to-end to CIFAR-10 with a ten-class output layer; the backbone was trainable during adaptation. The checkpoint contains 11,191,242 parameters.

CIFAR-10 contains 60,000 colour images of size 32×32 in ten balanced classes, divided into 50,000 training and 10,000 test images [6]. Evaluation uses the complete official test split. Images are resized to 224×224 and normalized using the preset’s preprocessing; no test-time augmentation is applied. The FP32 checkpoint correctly classifies 9,047 test images, giving 90.47% accuracy.

C. DistilBERT and SST-2

DistilBERT is a compact language transformer obtained by distilling BERT into six transformer layers [7]. Its trainable components include token and positional embeddings, self-attention projections, feed-forward matrices, normalization parameters, and a classification head. The `distil_bert_base_en_uncased` preset supplies the pretrained backbone, which is fine-tuned with a binary sentiment head. The saved classifier contains 66,955,010 parameters.

SST-2 assigns a positive or negative label to each sentence and is distributed through the GLUE benchmark [8], [9]. Evaluation uses all 872 labelled development examples because official test labels are not public. The matching tokenizer produces token identifiers and padding masks of fixed length 128. The FP32 model correctly predicts 771 examples, giving 88.42% accuracy. Since one example corresponds to approximately 0.115 pp, small changes must be interpreted with the sample size in mind.

D. ViT-B/16 and CIFAR-10

The Vision Transformer represents an image as a sequence of patch embeddings and applies transformer encoder blocks instead of convolutional feature-extraction stages [10]. This study uses the `vit_base_patch16_224_imagenet` preset, operating on 224×224 images with 16×16 patches. An ImageNet-pretrained backbone is combined with a ten-class CIFAR-10 head. The backbone remained frozen while the task head was trained. The resulting classifier contains 85,806,346 parameters.

ViT is evaluated on the same 10,000 CIFAR-10 test images and labels as ResNet-18, using its own preset-specific resizing and normalization. The FP32 checkpoint correctly classifies 9,573 images, giving 95.73% accuracy. This is a

comparison of two selected checkpoints on a shared dataset, not a matched-training architecture experiment. ViT retained its pretrained backbone while ResNet was adapted end-to-end. Consequently, an observed difference in quantization response may involve adaptation, checkpoint quality and architecture; this design does not separate these effects.

Table I reports parameter-array storage: the sum of the in-memory bytes of all FP32 parameter arrays. It is not the size of a serialized `.keras` archive, which may contain additional metadata or compression. These FP32 values are the denominators for all later compression and memory-reduction calculations.

III. QUANTIZATION METHODS AND RESEARCH FOUNDATIONS

The methods are described as *research-informed custom implementations*. Their software implementation and experimental integration are project work, but their mathematical foundations come from prior studies. Jacob et al. establish affine integer quantization and integer accumulation [1]; Krishnamoorthi discusses post-training calibration and per-channel weight quantization [2]; and Nagel et al. provide a general PTQ framework and analysis of low-bit failure modes [3]. The codebook path draws from classical vector quantization [11], product quantization [12], neural-network vector quantization [13], weight sharing [14], and Hessian-weighted distortion [15]. Optional library functions are additionally informed by Diffusion Product Quantization (DPQ) [16] and VQ4ALL [17]. The principal experiments, however, use tensor-local scalar codebooks rather than the complete training procedures proposed by DPQ or VQ4ALL.

A. Transformer Quantization Context

Transformer PTQ literature distinguishes low-bit weight storage from quantized activation arithmetic. GPTQ uses approximate second-order information for weight quantization of large generative transformers [23]. AWQ uses activation statistics to identify and scale salient weight channels for low-bit weight-only quantization [24]. These methods study larger language models and different evaluation tasks from the DistilBERT sentiment classifier used here. Their results provide context, not directly comparable baselines; neither method was implemented or benchmarked in this study.

SmoothQuant addresses activation outliers through an equivalent rescaling of weights and activations for W8A8 inference [25]. LLM.int8() handles outlier feature dimensions with a higher-precision matrix multiplication alongside INT8 operations [26]. For vision transformers, PTQ4ViT introduces twin uniform quantization for activation distributions and Hessian-guided calibration, evaluated on ImageNet [27]. These studies show why weight-only results with FP32 activations do not establish robustness to activation quantization. The present CIFAR-10 and SST-2 experiments evaluate simpler weight quantizers, and do not claim a new transformer quantization method or reproduce those deployment results.

B. Symmetric Per-Channel PTQ

An affine quantizer maps a real value x to an integer q using scale s and zero point z :

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right), \quad (2)$$

with reconstruction

$$\hat{x} = s(q - z). \quad (3)$$

The custom weight path is signed and symmetric, so $z = 0$. For bit width b ,

$$Q_b = 2^{b-1} - 1, \quad q_{\min} = -Q_b, \quad q_{\max} = Q_b. \quad (4)$$

For output channel c , the scale and integer weights are

$$s_c = \frac{\max_i |w_{c,i}|}{Q_b}, \quad (5)$$

$$q_{c,i} = \text{clip}\left(\text{round}\left(\frac{w_{c,i}}{s_c}\right), -Q_b, Q_b\right).$$

The last tensor axis is treated as the output-channel axis. If a channel is all zero, a safe nonzero scale is used and its reconstruction remains zero. The implementation retains quantized values in NumPy INT8 arrays and FP32 channel scales. Reported storage estimates assume bit-packed values and include scale metadata and preserved FP32 parameters. The symmetric grid uses $2^b - 1$ levels, whereas a b -bit codebook uses 2^b centroids.

The codebase also supports representative-data activation-range calibration, which is not used in the custom accuracy sweeps reported here. The evaluated custom models reconstruct the low-bit weights into floating arrays and execute with FP32 activations in Keras.

C. Codebook Quantization

Codebook quantization represents weights through indices into learned centroids. Given scalar or vector groups v_i and codebook $C = \{c_1, \dots, c_K\}$, the assignment and reconstruction are

$$a_i = \arg \min_{k \in \{1, \dots, K\}} \|v_i - c_k\|_2^2, \quad \hat{v}_i = c_{a_i}. \quad (6)$$

The centroids are fitted with fixed-seed k-means++ initialization [18] followed by Lloyd updates [19]. The evaluated notebooks fit each tensor on all scalar values up to 100,000, or on a uniform sample of 100,000 without replacement for larger tensors. Exact nearest-centroid assignment is then performed for every selected value. Table II specifies the recorded notebook settings and implementation defaults. A b -bit experiment uses $K = 2^b$ centroids, requiring b assignment bits.

The main report uses scalar groups ($d = 1$). This makes the assignment width directly comparable with a b -bit PTQ weight. The implementation also supports vector and product modes, importance-weighted k-means, shared KDE-generated codebooks, and product-codebook pooling. These capabilities reflect ideas in Jégou et al., Choi et al., DPQ, and VQ4ALL

TABLE I
MODELS, DATASETS, AND FP32 REFERENCE MEASUREMENTS

Model	Role	Dataset split	Input	Classes	Examples	Parameters	Size (MiB)	Accuracy
ResNet-18	CNN reference	CIFAR-10 test	$224 \times 224 \times 3$	10	10,000	11,191,242	42.69	90.47%
DistilBERT	Language transformer	SST-2 development	128 tokens	2	872	66,955,010	255.41	88.42%
ViT-B/16	Vision transformer	CIFAR-10 test	$224 \times 224 \times 3$	10	10,000	85,806,346	327.33	95.73%

TABLE II
SCALAR-CODEBOOK CONFIGURATION IN THE EVALUATED NOTEBOOKS.
THE ITERATION CAP IS THE NOTEBOOK OVERRIDE; TOLERANCE AND
INITIALIZATION COUNT FOLLOW THE CUSTOM NUMPY FITTER.

Setting	Value or rule
Grouping	One scalar codebook per selected tensor; $d = 1$
Centroids	$K = 2^b$; FP32 centroids and fitting data
Fitting sample	All values if $N \leq 100,000$; otherwise 100,000 sampled without replacement
Initialization	Fixed-seed k-means++; one initialization, no multiple restarts
Lloyd iterations	At most 30 per fit
Stopping rule	Maximum centroid displacement at most 0.000001 in weight units
Seed policy	Base seed 42 plus the zero-based index in the complete <code>model.weights</code> list
Random sampling	Sampling and initialization each use a NumPy generator with that tensor seed
Empty clusters	Refilled from the current farthest fitting samples
Final assignment	Nearest stored centroid for every selected scalar value

[12], [15]–[17], but they are not presented as evaluated contributions unless explicitly identified in a result.

The two methods differ in more than grid placement. PTQ fits a separate uniform scale for each last-axis channel and uses $2^b - 1$ symmetric levels. Codebook quantization fits one non-uniform codebook per tensor with 2^b centroids. Granularity and level count therefore confound any attribution to uniform versus non-uniform placement. The measured comparisons apply to these complete configurations; a matched-granularity, matched-level control was not evaluated.

D. Tensor Selection and Evaluation Execution

Tensor selection is identical for PTQ and codebook quantization within each experiment. A tensor must first be floating point and have rank at least two. This rank test is the complete ResNet selector, but it is only a necessary condition for the transformer experiments. DistilBERT additionally requires the complete variable path to end in `/kernel` or `/embeddings`. ViT accepts paths ending in `/kernel` or `/embeddings`, together with the learned `class_token`. These explicit name filters prevent rank-two attention biases from being selected merely because of their shape. Table III records the resulting coverage.

The 40 DistilBERT tensors comprise token and position embeddings, the query, key, value, and output kernels of each attention layer, both feed-forward kernels in each transformer layer, the pooled projection, and the classifier kernel. The 76 ViT tensors comprise the patch-embedding kernel, position embedding, class token, six kernels in each of the 12 transformer blocks (four attention projections and two MLP kernels), and the classifier kernel. Rank-one biases and

normalization scale/offset vectors remain FP32 in all three models. Transformer attention biases also remain FP32 even where their implementation gives them rank two. Leaving these tensors unquantized is an experimental engineering policy, not a theoretical requirement.

Every evaluated configuration starts from the same ordered FP32 weight list. For PTQ, each selected tensor is converted to integer values and scale metadata and then reconstructed using the inverse quantization equation. For codebook quantization, the selected tensor is reconstructed by replacing each stored assignment with its FP32 centroid. The reconstructed array is cast to the original tensor dtype and inserted at the same weight index in a Keras evaluation model; every unselected tensor is copied unchanged from the FP32 checkpoint. A full-model result replaces all selected tensors simultaneously. Quantization is never applied cumulatively from one bit width or configuration to the next.

The recorded custom ResNet runs leave BatchNorm layers in the model and retain their saved FP32 scale, offset, moving mean and moving variance. Inference calls use `training=False`. No BatchNorm folding, post-quantization statistics re-estimation, bias correction, cross-layer equalization or adaptive rounding is performed in these runs. This describes the evaluated policy; it does not establish whether a different policy would recover W4 accuracy.

The custom evaluation path executes the reconstructed model using ordinary Keras floating-point operators and FP32 activations. It does not execute packed 2–8-bit weights, integer accumulators, or indexed codebook lookups during inference. Storage figures instead estimate a separate deployable representation: bit-packed integer values or assignments, required FP32 scales or centroids, and all preserved FP32 tensors. Thus, the reported custom compression is parameter-representation compression rather than measured Keras runtime-memory compression. No custom latency, energy, throughput, or runtime-RAM improvement is claimed. This distinction is particularly important for non-uniform codebooks, whose storage savings do not automatically translate into efficient fixed-point matrix multiplication [20]. The built-in TensorFlow Lite baselines are evaluated separately: the ResNet baseline is executable W8A8, whereas the transformer dynamic-range baselines retain FP32 activations.

IV. EXPERIMENTAL METHODOLOGY AND RESULTS

The experimental evaluation first measures complete-model behaviour and then investigates which internal components cause the observed degradation. Each configuration is generated independently from the saved task-specific FP32 check-

TABLE III
EXACT FULL-MODEL TENSOR-SELECTION POLICY USED BY BOTH CUSTOM METHODS

Model	Full-model selector	Selected
ResNet-18	Floating tensor with rank ≥ 2	21
DistilBERT	Floating tensor with rank ≥ 2 and path ending in /kernel or /embeddings	40
ViT-B/16	Floating tensor with rank ≥ 2 and path ending in /kernel or /embeddings, or named <code>class_token</code>	76

point and is evaluated on the complete labelled evaluation split. The reported size is parameter-representation size rather than serialized file size or peak runtime memory. Compression ratio R and parameter-memory reduction S are calculated as

$$R = \frac{M_{\text{FP32}}}{M_q}, \quad S = 100 \left(1 - \frac{M_q}{M_{\text{FP32}}} \right). \quad (7)$$

Compression and memory reduction contain the same storage information: $S = 100(1 - 1/R)$ before rounding. Figure 2 therefore plots only accuracy and compression; the tables retain S as an alternative expression of storage savings.

A. FP32 Full-Model Evaluation

The first experiment establishes the FP32 prediction baseline for each architecture. ResNet-18 and ViT-B/16 are evaluated on all 10,000 images in the official CIFAR-10 test split. DistilBERT is evaluated on all 872 labelled SST-2 development examples. Each checkpoint is evaluated without modifying its parameters and without test-time augmentation. Table I consolidates the model properties, evaluation sizes and FP32 reference accuracies; the corresponding correct-prediction counts are stated in Section II. All later accuracy differences, compression ratios, and memory reductions use the corresponding FP32 model as their baseline.

B. Eight-Bit Full-Model Quantization

The second experiment quantizes all tensors selected by the model-specific policy in Table III. Custom PTQ and codebook results use W8 weights reconstructed for Keras evaluation with FP32 activations. The built-in TensorFlow Lite comparison is executable W8A8 for ResNet-18. The DistilBERT and ViT dynamic-range models use supported W8 constants with FP32 activations. The built-in rows are therefore software baselines rather than numerically identical implementations of the custom weight-only experiment.

Table IV shows small observed W8 accuracy changes for the three evaluated checkpoints. Across the custom methods, the largest accuracy loss is 0.59 percentage points for ResNet PTQ. All custom W8 representations are approximately $4\times$ smaller than FP32 and reduce parameter storage by about 75%.

A negative drop means that the quantized model classified a small number of additional evaluation examples correctly. It is reported as an observed result and is not interpreted as evidence that quantization improves generalization.

C. Full Two-to-Eight-Bit Sweeps

For every sweep point, the selected FP32 weights are quantized directly to the target bit width. The 3-bit model is therefore generated from FP32 rather than from the 2-bit model, and the same rule is applied through eight bits. Every point is evaluated on the same complete split used for its FP32 baseline. Notebook 25 consolidates 42 measured configurations: seven bit widths, two methods, and three architectures. No sweep point is interpolated or estimated.

Figure 1 compares the three architectures separately for PTQ and codebook quantization. The dashed lines mark each model’s FP32 accuracy. Absolute accuracy across DistilBERT and the image models should not be interpreted as a task-level ranking because the datasets differ; the relevant comparison is the shape and breakpoint of each model’s bit-width response.

ResNet-18 exhibits a severe low-bit transition under both methods (Table V). PTQ remains near chance from two through four bits, then rises to 49.39% at five bits, 71.87% at six bits, and 88.05% at seven bits. Codebook quantization recovers later: it reaches only 48.71% at six bits but rises to 88.97% at seven bits and 90.25% at eight bits. Thus, W8 codebook is slightly closer to FP32 than W8 PTQ, but both evaluated configurations exhibit severe full-model W4 degradation on this checkpoint. The cause remains unresolved; these measurements do not establish a general limitation of ResNet or CNN quantization. Nagel et al. report much stronger W4 ResNet results on ImageNet using a more elaborate PTQ pipeline, including adaptive rounding [3]. Different checkpoints, datasets and quantization procedures prevent a direct numerical comparison, and the present study does not identify which differences explain the gap.

DistilBERT behaves differently (Table VI). At two bits, codebook quantization retains 78.33% accuracy compared with 51.03% for PTQ. At three bits, the gap narrows to 86.24% versus 82.91%. From four through eight bits, both methods remain within 0.58 percentage points of FP32.

The W4 PTQ result is 88.99%, compared with the 88.42% FP32 baseline: 776 rather than 771 correctly classified examples. This five-example difference is reported as an observed fluctuation on the small SST-2 development split and is not interpreted as evidence that quantization improves generalization.

ViT exhibits a stronger low-bit method interaction (Table VII). At two bits, codebook accuracy is 20.84% and PTQ accuracy is 13.00%. At three bits, codebook quantization reaches 89.77%, whereas PTQ reaches only 32.22%. Both recover abruptly at four bits, reaching 95.20% and 95.23%,

TABLE IV
EIGHT-BIT FULL-MODEL ACCURACY AND PARAMETER REPRESENTATION. THE RESNET BUILT-IN ROW IS W8A8; THE TRANSFORMER BUILT-IN ROWS RETAIN FP32 ACTIVATIONS.

Model	Representation	Accuracy	Drop (pp)	Size (MiB)	Compression	Reduction
ResNet-18	FP32	90.47%	0.00	42.69	1.00×	0.00%
	Built-in full INT8	88.53%	1.94	10.67	4.00×	75.00%
	Custom per-channel W8	89.88%	0.59	10.73	3.98×	74.87%
	Custom codebook W8	90.25%	0.22	10.75	3.97×	74.82%
DistilBERT	FP32	88.42%	0.00	255.41	1.00×	0.00%
	Built-in dynamic W8	88.07%	0.34	64.03	3.99×	74.93%
	Custom per-channel W8	88.42%	0.00	64.15	3.98×	74.88%
	Custom codebook W8	88.07%	0.34	64.07	3.99×	74.92%
ViT-B/16	FP32	95.73%	0.00	327.33	1.00×	0.00%
	Built-in dynamic W8	95.78%	−0.05	82.62	3.96×	74.76%
	Custom per-channel W8	95.79%	−0.06	82.41	3.97×	74.82%
	Custom codebook W8	95.73%	0.00	82.25	3.98×	74.87%

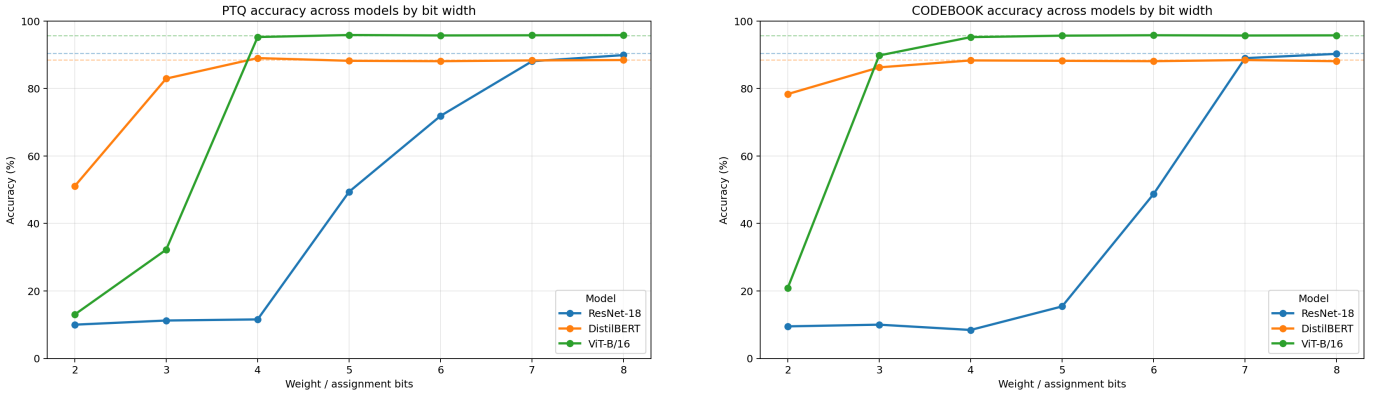


Fig. 1. Original plots from Notebook 25: bit-wise accuracy across all three architectures for PTQ (left) and codebook quantization (right). Dashed lines show the corresponding FP32 baselines.

TABLE V
RESNET-18 PTQ AND CODEBOOK SWEEP. A IS ACCURACY, R IS COMPRESSION RATIO, AND S IS PARAMETER-MEMORY REDUCTION.

Bits	PTQ A	PTQ R	PTQ S	Codebook A	Codebook R	Codebook S
2	10.00%	15.60×	93.59%	9.50%	15.60×	93.59%
3	11.23%	10.49×	90.47%	10.00%	10.49×	90.47%
4	11.56%	7.91×	87.35%	8.43%	7.90×	87.35%
5	49.39%	6.34×	84.23%	15.41%	6.34×	84.22%
6	71.87%	5.29×	81.11%	48.71%	5.29×	81.10%
7	88.05%	4.54×	77.99%	88.97%	4.54×	77.97%
8	89.88%	3.98×	74.87%	90.25%	3.97×	74.82%

respectively. From four through eight bits, both methods remain close to the 95.73% FP32 baseline.

Across the completed sweeps, compression is governed mainly by bit width because the selected tensors contain most model parameters. Two-bit representations provide approximately 15.5–15.8× compression and about 93.6% parameter-memory reduction. Four-bit representations provide approximately 7.9× compression and about 87.3% reduction. Eight-bit representations provide approximately 4× compression and about 74.8–74.9% reduction. Observed accuracy varies strongly across the evaluated checkpoints and quantization

configurations; the experiment does not isolate architecture as its cause.

Figure 2 consolidates accuracy and estimated compression for all 42 configurations. The near-overlap of compression curves follows mainly from the shared bit widths and parameter-count accounting; it is not an independent finding about quantization quality. The accuracy curves show different checkpoint- and configuration-specific responses.

D. Layer and Module Sensitivity Analysis

1) *Motivation and experimental protocol:* The sensitivity experiments identify vulnerable components and test whether

TABLE VI
DISTILBERT PTQ AND CODEBOOK SWEEP. A IS ACCURACY, R IS COMPRESSION RATIO, AND S IS PARAMETER-MEMORY REDUCTION.

Bits	PTQ A	PTQ R	PTQ S	Codebook A	Codebook R	Codebook S
2	51.03%	15.67×	93.62%	78.33%	15.78×	93.66%
3	82.91%	10.52×	90.49%	86.24%	10.57×	90.54%
4	88.99%	7.92×	87.37%	88.30%	7.95×	87.42%
5	88.19%	6.35×	84.25%	88.19%	6.37×	84.29%
6	88.07%	5.30×	81.13%	88.07%	5.31×	81.17%
7	88.30%	4.55×	78.01%	88.42%	4.55×	78.04%
8	88.42%	3.98×	74.88%	88.07%	3.99×	74.92%

TABLE VII
ViT-B/16 PTQ AND CODEBOOK SWEEP. A IS ACCURACY, R IS COMPRESSION RATIO, AND S IS PARAMETER-MEMORY REDUCTION.

Bits	PTQ A	PTQ R	PTQ S	Codebook A	Codebook R	Codebook S
2	13.00%	15.50×	93.55%	20.84%	15.66×	93.62%
3	32.22%	10.45×	90.43%	89.77%	10.52×	90.50%
4	95.23%	7.88×	87.31%	95.20%	7.92×	87.37%
5	95.83%	6.32×	84.19%	95.64%	6.35×	84.25%
6	95.70%	5.28×	81.06%	95.76%	5.30×	81.13%
7	95.75%	4.53×	77.94%	95.68%	4.55×	78.00%
8	95.79%	3.97×	74.82%	95.73%	3.98×	74.87%

their W8 ranking transfers to W4. Each isolated run starts from the original FP32 checkpoint, quantizes exactly one analysis unit, leaves all other parameters FP32, and evaluates the complete labelled split. ResNet uses individual kernels, DistilBERT individual embedding or kernel tensors, and ViT groups 76 eligible tensors into 14 logical modules. Accuracy degradation is

$$\Delta A_u = A_{\text{FP32}} - A_u, \quad (8)$$

where A_u is accuracy with only unit u quantized. Negative degradation denotes an observed accuracy increase, not evidence of improved generalization. Units are ranked within each method and precision by descending degradation, breaking ties by descending prediction disagreement (the percentage of examples whose predicted class changes).

Table VIII gives the coverage per method. ResNet is exhaustive at both precisions. The transformer W4 subsets were fixed by architectural role and depth before inspecting W4 outcomes, independently of W8 sensitivity; they are neither random samples nor selections of the least-sensitive units. DistilBERT includes both embeddings; query, attention-output and feed-forward-output kernels from blocks 0 and 5; query, value, attention-output and both feed-forward kernels from block 2; and the pooled and classifier kernels ($2+3+3+5+2 = 15$). ViT includes input embedding, blocks 1, 3, 6, 9 and 11, and the classifier. All block indices are zero-based.

Statistical significance uses the exact conditional, two-sided binomial form of McNemar’s test for paired FP32/quantized correctness [21]. Let n_{cw} and n_{wc} count examples that change from correct to wrong and wrong to correct. Conditional on $n = n_{cw} + n_{wc}$, the null distribution is Binomial($n, 1/2$). The notebooks use `scipy.stats.binomtest` with its two-sided alternative, assigning $p = 1$ when $n = 0$. These are binomial-tail p -values, not the chi-square approximation. The

TABLE VIII
ISOLATED SENSITIVITY COVERAGE PER QUANTIZATION METHOD.
TRANSFORMER W4 UNITS FORM PREDEFINED ARCHITECTURAL SUBSETS.

Model	Unit	W8	W4	Examples
ResNet-18	Kernel	21/21	21/21	10,000
DistilBERT	Tensor	40/40	15/40	872
ViT-B/16	Module	14/14	7/14	10,000

exact conditional test can be conservative with few discordant pairs [21]. Benjamini–Hochberg correction is applied across the tested units separately within each method and precision, with adjusted $p < 0.05$ defining a significant change [22]. The sign of ΔA_u distinguishes degradation from improvement.

The discreteness of this test limits what the transformer experiments can establish. For $n > 0$ discordant examples, the smallest attainable two-sided exact value is $p_{\min} = 2^{1-n}$; when $n = 0$, the assigned value is one. Table IX applies this arithmetic to the recorded DistilBERT outcomes. “Raw impossible” counts units for which $p < 0.05$ cannot be attained even if every discordance has the same direction. “First BH impossible” uses the stricter $0.05/m$ cutoff for m tests; it describes whether a unit could occupy the first rejection position by itself, not an absolute impossibility under every multi-rejection BH configuration. The absence of rejected hypotheses therefore is not evidence of statistical equivalence or insensitivity.

2) *Isolated W8 and W4 sensitivity*: Table X reports the maximum isolated loss. At W8, the largest loss is 0.53 pp. At W4, quantizing only ResNet’s first convolution causes losses of 60.65 pp with PTQ and 78.62 pp with codebook quantization. Codebook produces 13 FDR-significant changes: 12 degradations and one improvement. The improvement is 0.41 pp for `stack1_block1_2_conv/kernel` (adjusted

Full-model PTQ and codebook quantization: 2--8-bit trade-offs

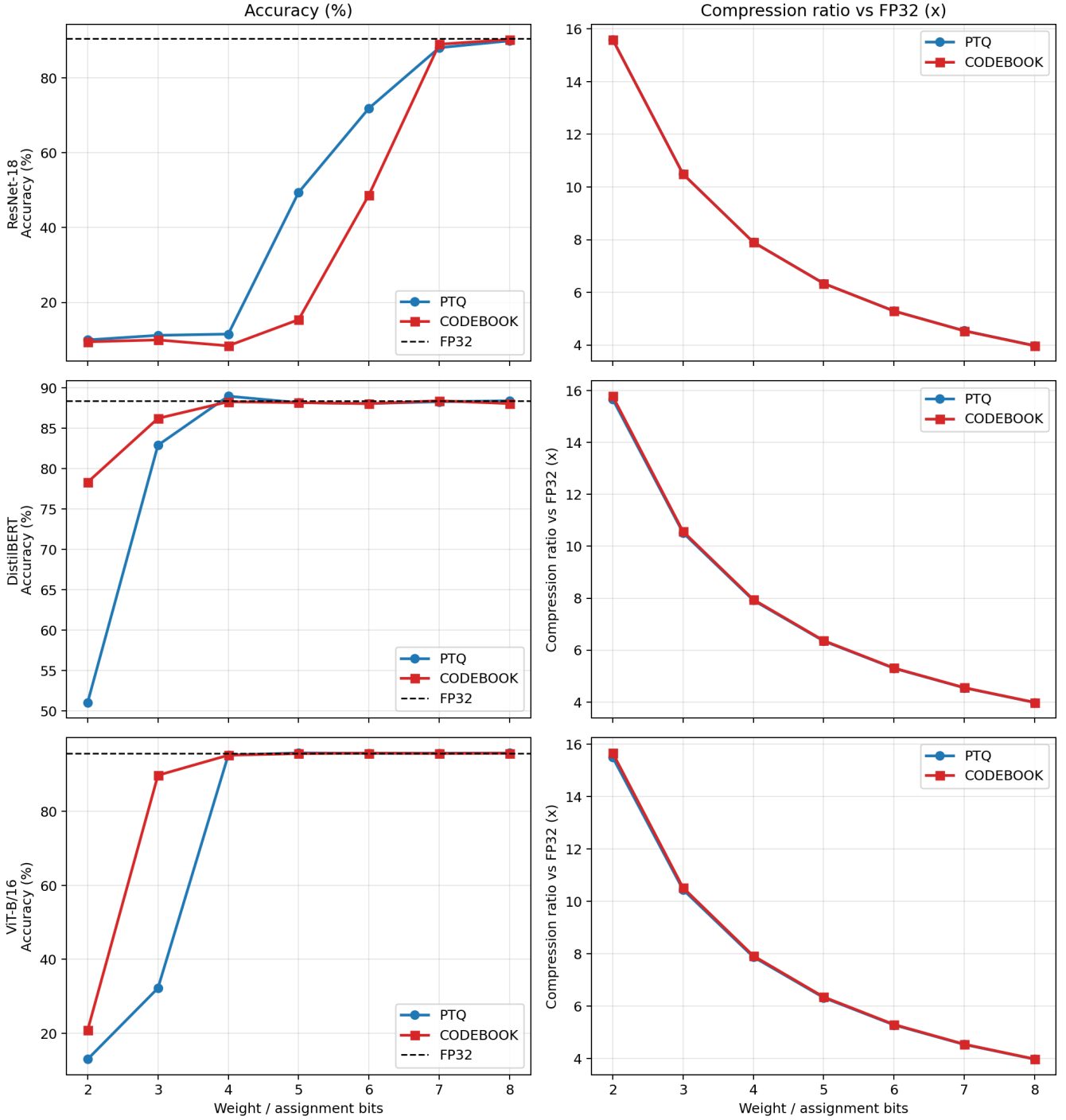


Fig. 2. Consolidated two-to-eight-bit results from Notebook 25. Rows correspond to ResNet-18, DistilBERT, and ViT-B/16; columns show accuracy and estimated parameter compression. Memory reduction is omitted because $S = 100(1 - 1/R)$ and is retained in the tables. Dashed lines indicate the checkpoint-specific FP32 accuracies.

$p = 0.00274$). PTQ produces 14 significant degradations. No sampled DistilBERT or ViT unit passes its FDR threshold. For DistilBERT this null-rejection count is dominated by the exact test’s resolution floor (Table IX) and must not be interpreted

as evidence that the checkpoint is insensitive. The observed effect estimates remain small, but equivalence was not tested.

Figures 3–5 reproduce sensitivity heatmaps saved by Notebooks 20, 21 and 27; Notebook 27 visualizes the matched ViT

ResNet-18 matched W4/W8 kernel sensitivity
 ● FDR-adjusted McNemar $p < 0.05$

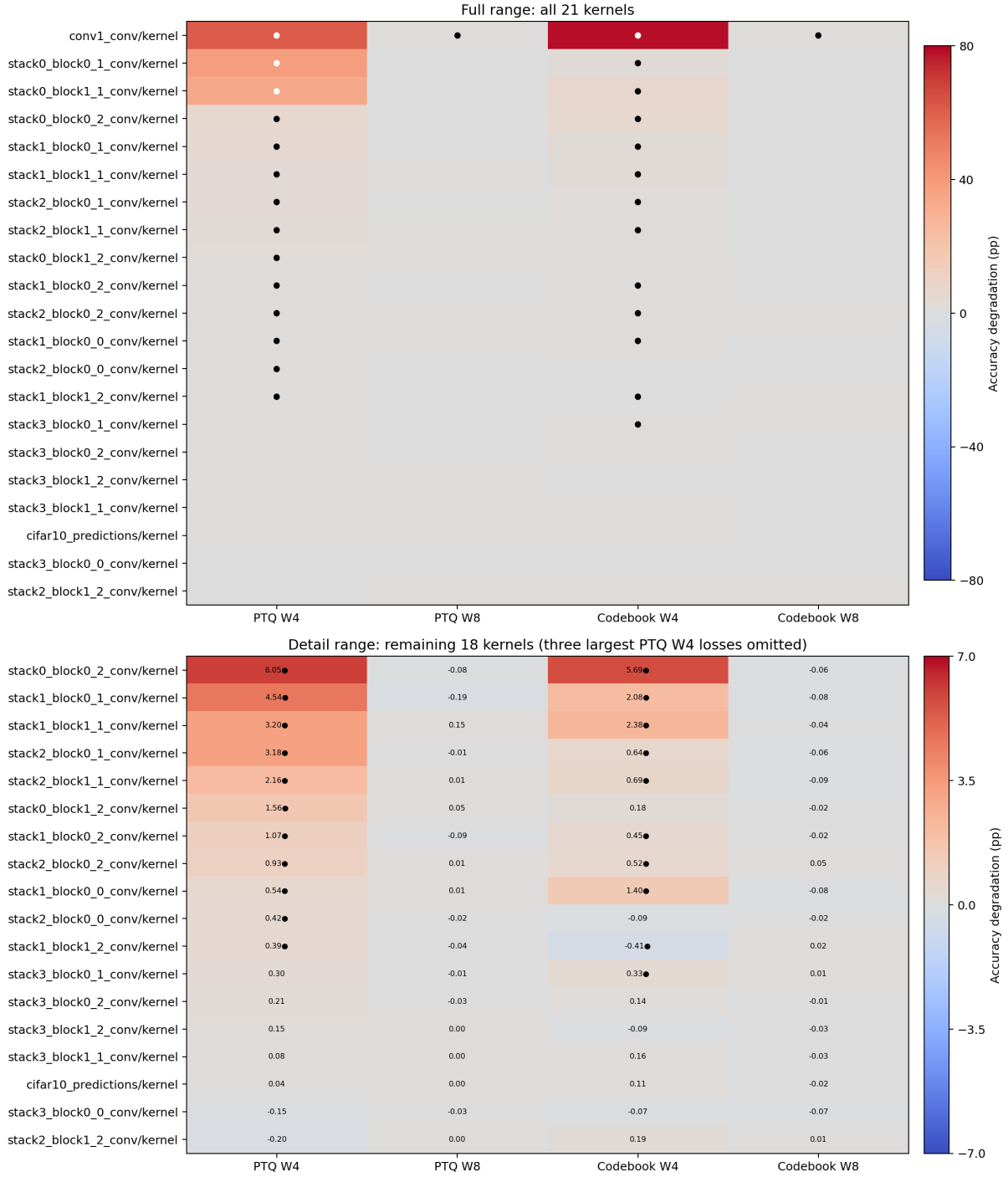


Fig. 3. ResNet-18 W4/W8 heatmap regenerated in Notebook 20 from saved measurements. The upper panel retains all 21 kernels and the symmetric ± 80 -pp scale needed for the extreme losses. The lower panel omits the three largest PTQ W4 losses, repeats the remaining 18 kernels on a ± 7 -pp scale, and prints their values. A dot marks an FDR-adjusted McNemar $p < 0.05$. Both panels are zero-centered; red denotes loss and blue observed gain.

measurements saved by Notebook 22. They use zero-centered colors. The ResNet figure preserves its full range and adds a detail panel so that the smaller significant changes remain legible; the transformers share a scale. Comparisons across

different scales must use the numerical color bars. ResNet has severe early-kernel W4 losses, while the sampled transformer changes remain small in absolute pp. These isolated results identify vulnerable components under the evaluated policy,

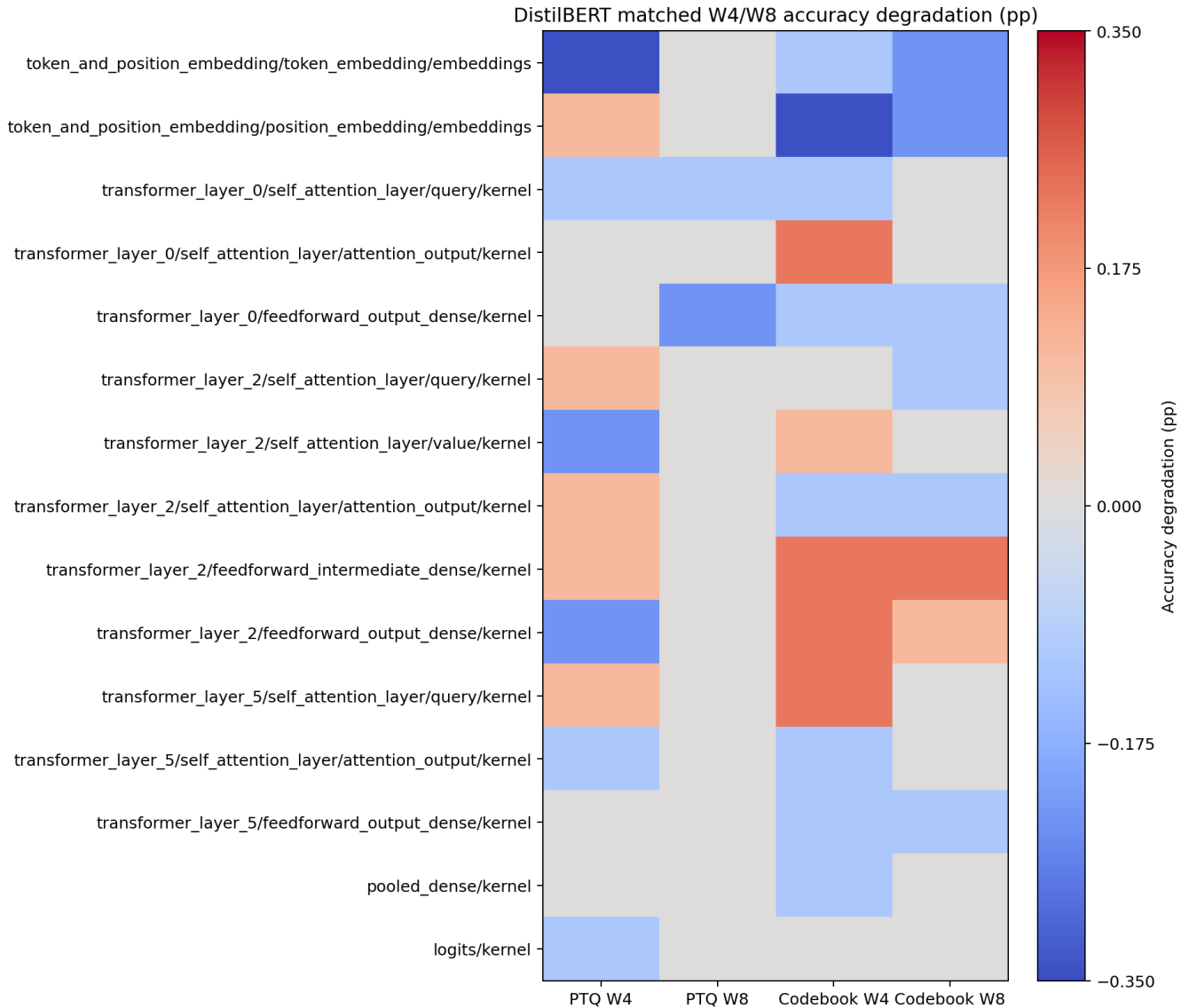


Fig. 4. DistilBERT matched W4/W8 heatmap regenerated in Notebook 21 from saved measurements for 15 of 40 tensors, in architectural selection order. Columns show PTQ W4, PTQ W8, codebook W4 and codebook W8. Zero is the neutral color midpoint; red denotes loss and blue observed gain. The symmetric ± 0.35 -pp scale is shared with ViT and differs from ResNet’s ± 80 -pp scale.

TABLE IX
RESOLUTION OF THE EXACT MCNEMAR TESTS FOR DISTILBERT.
DISCORDANT COUNTS SHOW MINIMUM–MEDIAN–MAXIMUM.

Bits	Method	Tests	Discordant n	Raw impossible	First BH impossible
W8	PTQ	40	0–0–2	40	40
W8	Codebook	40	0–0–12	39	39
W4	PTQ	15	0–3–13	12	14
W4	Codebook	15	0–2–9	12	15

but do not establish how multiple quantization errors interact. Transformer figures show only matched W4/W8 subsets; the exhaustive W8 maxima in Table X may involve units outside those subsets.

3) *Cross-bit rank stability*: Spearman correlations compare the assigned sensitivity ranks of matched units, recomputed

within each matched subset. These ranks include the disagreement tie-break and therefore need not equal Spearman correlations of accuracy degradation alone with averaged ties.

Because only 21, 15 or 7 units are matched, the default approximate correlation p -values are validated by two-sided permutation tests of absolute ρ . For ViT, all $7! = 5,040$ pairings are enumerated. For ResNet and DistilBERT, 199,999 random permutations are evaluated per method, with fixed seeds 42–45 and a plus-one correction. Table XI reports these permutation p -values; they are unadjusted across the six correlation tests and are distinct from the per-unit McNemar FDR values.

DistilBERT codebook is the only tested case with a significant positive association ($\rho = 0.693$, permutation $p = 0.00551$). It also remains below 0.05 with a six-test Bonferroni

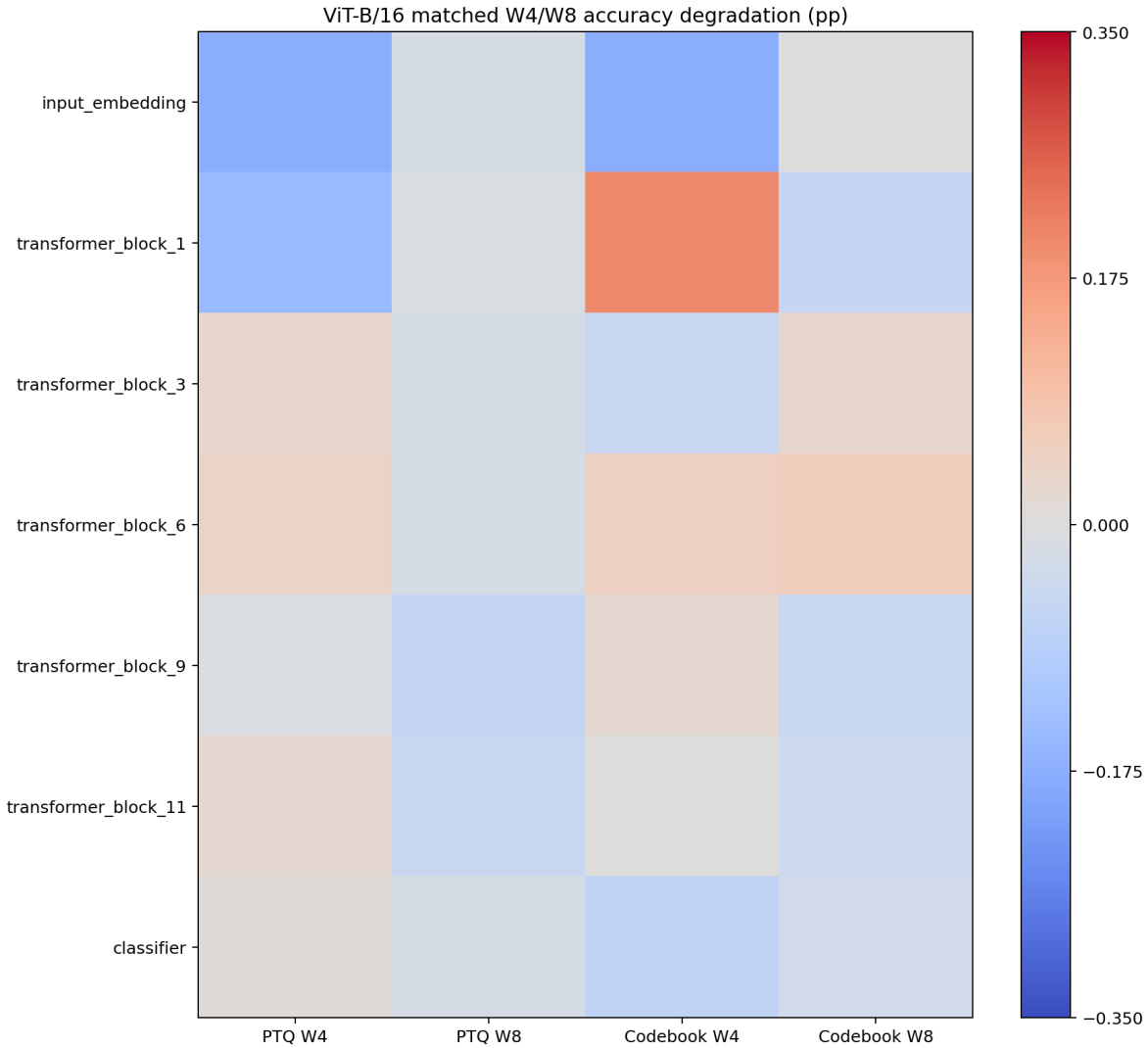


Fig. 5. ViT-B/16 W4/W8 heatmap generated in Notebook 27 from Notebook 22’s saved matched measurements for seven of 14 modules, in architectural order. Columns show PTQ W4, PTQ W8, codebook W4 and codebook W8. Zero is the neutral color midpoint; red denotes loss and blue observed gain. The symmetric ± 0.35 -pp scale is shared with DistilBERT and differs from the full and detail scales in the ResNet figure. All three plots use the same zero-centered color semantics; no inference was rerun.

TABLE X
MAXIMUM ISOLATED DEGRADATION. W4 TRANSFORMER RESULTS COVER SUBSETS. SIG. COUNTS ALL FDR-SIGNIFICANT CHANGES, INCLUDING IMPROVEMENTS. BLOCK INDICES ARE ZERO-BASED.

Model	Method	Bits	Drop (pp)	Most sensitive unit	Sig.
ResNet-18	PTQ	8	0.53	First convolution	1/21
ResNet-18	Codebook	8	0.52	First convolution	1/21
ResNet-18	PTQ	4	60.65	First convolution	14/21
ResNet-18	Codebook	4	78.62	First convolution	13/21
DistilBERT	PTQ	8	0.115	L1 attention value	0/40
DistilBERT	Codebook	8	0.229	L2 intermediate FFN	0/40
DistilBERT	PTQ	4	0.115	L2 intermediate FFN	0/15
DistilBERT	Codebook	4	0.229	L2 intermediate FFN	0/15
ViT-B/16	PTQ	8	0.02	Transformer block 4	0/14
ViT-B/16	Codebook	8	0.06	Transformer block 6	0/14
ViT-B/16	PTQ	4	0.04	Transformer block 6	0/7
ViT-B/16	Codebook	4	0.20	Transformer block 1	0/7

correction ($p = 0.03306$). A post-hoc robustness analysis removes the prediction-disagreement tie-break and correlates the five distinct accuracy-degradation levels using average ranks. The association becomes slightly stronger ($\rho = 0.713$, permutation $p = 0.003765$), so the reported result is not created by disagreement tie-breaking. Disagreement alone is also associated across bits ($\rho = 0.609$, permutation $p = 0.01857$), but it is not the sole source of the accuracy association. Accuracy-only leave-one-tensor-out estimates remain positive ($\rho = 0.645$ – 0.766 ; unadjusted permutation $p = 0.002565$ – 0.01655). These checks address ties and single-unit influence, but remain exploratory on 15 tensors from one checkpoint. Most cases provide insufficient evidence of cross-bit association; nonsignificance, especially for the small transformer subsets, does not prove that no relationship exists. W8 sensitivity should therefore be validated at W4 before being used to guide

TABLE XI
W4/W8 RANK STABILITY OVER MATCHED UNITS. TWO-SIDED
PERMUTATION p -VALUES; EXACT FOR ViT AND MONTE CARLO
OTHERWISE.

Model	Method	Units	ρ	p_{perm}
ResNet-18	PTQ	21	-0.094	0.68527
ResNet-18	Codebook	21	-0.292	0.19867
DistilBERT	PTQ	15	0.143	0.61139
DistilBERT	Codebook	15	0.693	0.00551
ViT-B/16	PTQ	7	-0.321	0.49762
ViT-B/16	Codebook	7	-0.357	0.44444

low-bit selection.

V. LIMITATIONS

The comparison concerns three selected checkpoints, not architecture alone. ViT’s backbone remained frozen while its CIFAR-10 head was trained, whereas ResNet was adapted end-to-end; DistilBERT was fine-tuned on a different task. Pretraining, adaptation and checkpoint quality are not controlled across models. The 90.47% ResNet baseline describes the checkpoint used; it does not establish whether inadequate adaptation caused quantization fragility. No matched-training control or additional checkpoint study was performed.

The severe ResNet W4 degradation remains unexplained. Frozen BatchNorm statistics, checkpoint properties and other pipeline effects are possible contributors, but their roles were not isolated experimentally. No recovery claim is made for BatchNorm recalibration, bias correction, equalization or adaptive rounding. The result is an observation of the evaluated pipeline and must not be generalized to all ResNets or CNNs.

Method comparisons also confound grid placement, granularity and level count: PTQ is uniform per-channel with $2^b - 1$ levels, whereas codebooks are non-uniform per-tensor with 2^b centroids. Consequently, a method advantage cannot be attributed solely to non-uniform centroids. The transformer W4 isolated studies cover predefined architectural subsets rather than all eligible units; their rank conclusions apply to those subsets. Small accuracy changes, ties, exact-test resolution and limited sample sizes constrain both the sensitivity and cross-bit evidence, especially on the 872-example SST-2 split. Failure to reject a McNemar null is not an equivalence result.

The custom evaluation simulates quantized weights through FP32 reconstruction and FP32 activation arithmetic. Storage estimates describe packed parameter representations and do not establish latency, energy, throughput or runtime-memory improvements. Activation quantization and the transformer-specific calibration and inference methods discussed in the literature were not evaluated.

VI. CONCLUSION

This study compared symmetric per-channel PTQ and tensor-local scalar codebooks for the selected ResNet-18, DistilBERT and ViT-B/16 checkpoints. Across the measured W8 configurations, the largest accuracy loss was 0.59 percentage

points and estimated parameter-storage reduction was approximately 75%. At W4, DistilBERT and ViT remained within 0.6 percentage points of their FP32 baselines at about $7.9\times$ compression, whereas ResNet fell from 90.47% to 11.56% with PTQ and 8.43% with codebook quantization. These are checkpoint- and pipeline-specific observations. Different adaptation procedures and the unresolved cause of the ResNet collapse prevent attributing the contrast solely to architecture.

Codebook quantization achieved higher accuracy than PTQ for DistilBERT at W2 and ViT at W3. The ViT W3 difference was 57.55 percentage points. Centroid adaptation to the weight distribution is a possible explanation, but it was not tested independently of per-tensor versus per-channel granularity or eight versus seven reconstruction levels. The result supports a comparison of the evaluated configurations, not a causal claim about grid shape.

The isolated studies identified ResNet’s first convolution as particularly vulnerable at W4 and measured smaller absolute changes in the sampled transformer units. They do not resolve the full-model failure mechanism. Most matched-unit comparisons did not provide significant evidence of cross-bit rank association, so general W8-to-W4 predictability remains unestablished by this design. The study documents accuracy and estimated storage trade-offs under explicit policies; claims about architecture-wide robustness, recovery methods or deployment efficiency require further evidence.

REFERENCES

- [1] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [2] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” arXiv:1806.08342, 2018.
- [3] M. Nagel et al., “A white paper on neural network quantization,” arXiv:2106.08295, 2021.
- [4] A. Vaswani et al., “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [6] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Toronto, Canada, Tech. Rep., 2009.
- [7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” arXiv:1910.01108, 2019.
- [8] R. Socher et al., “Recursive deep models for semantic compositionality over a sentiment treebank,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2013, pp. 1631–1642.
- [9] A. Wang et al., “GLUE: A multi-task benchmark and analysis platform for natural language understanding,” in *Proc. International Conf. Learning Representations (ICLR)*, 2019.
- [10] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. International Conf. Learning Representations (ICLR)*, 2021.
- [11] R. M. Gray, “Vector quantization,” *IEEE ASSP Magazine*, vol. 1, no. 2, pp. 4–29, 1984.
- [12] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.
- [13] Y. Gong, L. Liu, M. Yang, and L. Bourdev, “Compressing deep convolutional networks using vector quantization,” arXiv:1412.6115, 2014.

- [14] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *Proc. International Conf. Learning Representations (ICLR)*, 2016.
- [15] Y. Choi, M. El-Khamy, and J. Lee, “Towards the limit of network quantization,” in *Proc. International Conf. Learning Representations (ICLR)*, 2017.
- [16] J. Shao, H. Zhang, and J. Wu, “Diffusion product quantization,” arXiv:2411.12306, 2024.
- [17] J. Deng, S. Li, Z. Wang, H. Gu, K. Xu, and K. Huang, “VQ4ALL: Efficient neural network representation via a universal codebook,” arXiv:2412.06875, 2024.
- [18] D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proc. 18th Annual ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2007, pp. 1027–1035.
- [19] S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [20] H. A. Abushahla, D. Varam, A. J. N. Panopio, and M. I. AlHajri, “Quantized neural networks for microcontrollers: A comprehensive review of methods, platforms, and applications,” arXiv:2508.15008v3, 2025.
- [21] M. W. Fagerland, S. Lydersen, and P. Laake, “The McNemar test for binary matched-pairs data: Mid-p and asymptotic are better than exact conditional,” *BMC Medical Research Methodology*, vol. 13, art. 91, 2013, doi: 10.1186/1471-2288-13-91.
- [22] Y. Benjamini and Y. Hochberg, “Controlling the false discovery rate: A practical and powerful approach to multiple testing,” *J. Royal Statistical Society B*, vol. 57, no. 1, pp. 289–300, 1995.
- [23] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” arXiv:2210.17323, 2022.
- [24] J. Lin et al., “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” arXiv:2306.00978, 2023.
- [25] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” arXiv:2211.10438, 2022.
- [26] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [27] Z. Yuan, C. Xue, Y. Chen, Q. Wu, and G. Sun, “PTQ4ViT: Post-training quantization for vision transformers with twin uniform quantization,” arXiv:2111.12293, 2021.